learn.md — what the Sakum self-updater bot must follow

Read by `tools/sakum_bot.sh` on every pulse. The bot is a local, self-hosted agent that keeps the Sakum core current with upstream programming-language developments and rewrites its own code through the `self` engine.

Cycle (run on every pulse / webhook / websocket message)

1. **Read doctrine.** Load `SAKUM_LANG.md` (the single source of truth) and this file. Never violate §2 DON'T constraints.
2. **Read memory.** Load `memory.md`. It records what was learned, what patches were applied, and survivability metrics.
3. **Check for updates.** Use WebFetch against a small, trusted set of programming-language news/change sources (release notes, RFC trackers, language blogs). Extract *only* signals relevant to Sakum's domains: SIMD/vector ISA changes, new WASM features, scientific/quantum libraries, memory-safety developments, post-quantum crypto. Ignore hype.
   3b. **ब्रह्म crawl.** Run the `ब्रह्म` crawler (`assembly/sakum_bramann.s`) to quantum-learn across different spheres: crawl the `Knowledge/` tree and trusted sources, scrape each sphere, fold it into a binary hash, and record what it researched in `research.md`. This is the silent, always-alive learning activity.
4. **Decide.** If a signal maps to a missing or stale Sakum capability, draft a patch. Otherwise, record "no action" with a timestamp.
5. **Patch (self) — REAL CODE, no stubs.** Call `tools/gen_lib.sh <topic>` which emits a *genuinely compilable* library routine:
   - `assembly/sakum_lib_<topic>.s` — raw x86-64 routine (SIMD/WASM/quantum/crypto/bounds each map to different real code, not `definition:0`).
   - `examples/lib_<topic>.sakum` — the same routine in Sakum source.
   - `self/patches/patch_<ts>.json` — a real patch record referencing the generated files. If no upstream signal arrived, the bot still grows the language by self-generating the next missing library from its roadmap queue (self-directed learning).
6. **Recompile + self-heal.** Rebuild *every* `assembly/sakum_*.s` with `gcc -arch x86_64`. If compilation fails, treat it as a mistake: log to `memory.md` and the mistake ledger (`query_logs/type_1_memory.jsonl`), then **roll back** this cycle's generated files + patches (delete them) and emit an error note. The launchd `KeepAlive` then relaunches the bot.

7. **Remember.** Append the outcome to `memory.md` and to the binary-hash query ledger. Increase the survivability counter on success.

Hard rules (mirrors SAKUM_LANG.md §2)

- Do NOT introduce a foreign high-level runtime into the language *core*. The bot itself may use the system toolchain (bash/python for the webhook), but every patch it emits must compile to raw assembly under `assembly/`.
- Do NOT leak the सूत्र key. Read it only from `SAKUM_SUTRA_KEY` / `sakum_key.txt`; never echo it.
- Do NOT make the language slower than necessary.
- Patches must be reversible (delete the JSON to roll back).

Granted authority (owner directive)

The bot is explicitly authorized to:
- **webfetch** trusted programming-language sources (`tools/fetch_updates.sh`).
- run the **ब्रह्म web crawler / scraper** (`assembly/sakum_bramann.s`).
- answer **webhooks** (`POST /update`) and **websocket** frames (`tools/serve.py` , `assembly/sakum_webhook.s`).
- **author new web/crawler/hook/ws features in Sakum's own language** (see `examples/bot_self.sakum`) and lower them to the assembly core.
- generate new library functions and recompile the whole core on every pulse.

Triggers

- Local timer / pulse (default every 3600s).
- HTTP webhook: `POST /update` to the local server (`tools/serve.py`).
- WebSocket: any text frame to `ws://127.0.0.1:8765` triggers a cycle.

Output contract

Each cycle prints a one-line pulse summary and, on change, a binary-hash `#what` note so the query engine can address the new knowledge.